

Lab 5: Keypad Scanning and Display

Microprocessors (010153521) | Semester 1/2026

Duration: 3 hours (in-lab) | **Topic:** Multiplexed keypad input scanning on the Arduino Uno R3, decoded and shown on a single 7-segment digit in AVR assembly – the input-side counterpart to Lab 4’s multiplexed display output

Objectives

- Understand **matrix keypad scanning** as multiplexed *input* – the mirror image of Lab 4’s multiplexed *output* to a 2-digit display.
- Wire a 4x4 matrix keypad and a single 7-segment digit to the Arduino Uno R3.
- Write AVR assembly that drives one row **LOW** at a time, polls all four columns (internal pull-ups, idle **HIGH**), and decodes the resulting (row, column) pair into a key value.
- Debounce keypresses and display the result – reusing Lab 4-1’s DIGIT_TO_SEG subroutine *unchanged*.

Quick Reference: Pin Assignment

Port bits	Uno pins	Direction	Function
PB0-PB3	D8-D11	Output	Keypad rows R1-R4 (driven one at a time, active low)
PD2-PD5	D2-D5	Input, pull-up	Keypad columns C1-C4 (idle high ; pulled low through a pressed key when its row is driven low)
PC0-PC5	A0-A5	Output	Segments a,b,c,d,e,f
PD6	D6	Output	Segment g

This is the *same* segment bit order and hex table as Lab 4-1 – only segment **g**’s physical pin moved, from PD2 to PD6, because PD2-PD5 are now the keypad’s column inputs. There is no digit-select transistor this time: a single digit’s common cathode goes straight to **GND**.

Quick Reference: How Matrix Scanning Works

A 4x4 keypad packs 16 keys into just 8 wires – 4 rows + 4 columns – because each key does nothing but bridge *one* row wire to *one* column wire when pressed; there is no other connection inside the keypad. Driving all 4 rows at once would make every key on a pressed column look identical, so scanning removes the ambiguity: drive exactly *one* row **LOW** (the other three stay **HIGH**), read all 4 columns, and only trust what you read while that one row is the low one. Cycle through all 4 rows, fast enough that a keypress during any row’s turn gets caught.

	C1	C2	C3	C4
R1	1	2	3	A
R2	4	5	6	B
R3	7	8	9	C
R4	*	0	#	D

This is the standard layout for most inexpensive 4x4 membrane keypads – **confirm it against your physical keypad’s printed labels** before trusting any decoded output; wiring order varies between manufacturers. Only 0-9 have a segment code from Lab 4-1’s table; for A,B,C,D,*,#, just display a dash (segment **g** only) as a “not a digit” indicator – no new

segment codes needed.

Quick Reference: Your Individual Target

As in Labs 3 and 4-1, derive your personal target from your student ID:

$$D = \text{sum of every digit in your student ID}, \quad t_{\text{debounce}} = 15 \text{ ms} + (D \bmod 10) \times 1 \text{ ms}$$

Worked example (a made-up ID, do not reuse): ID 1029384756123 $\rightarrow D = 51$. Then $t_{\text{debounce}} = 15 + (51 \bmod 10) \times 1 = 15 + 1 = 16 \text{ ms}$.

Bill of Materials

Qty	Part	Notes
1	Arduino Uno R3 (from Lab 3/4)	Supplies power, ground, and all signal pins
1	4x4 matrix keypad module (8-pin)	R1-R4, C1-C4
1	7-segment display, common cathode	Single digit – no transistor needed
7	Resistor, 330 Ω	Segment current limiting, a-g
1	Breadboard + jumper wires	

Building the Circuit

1. **Rows.** Wire keypad R1-R4 to PB0-PB3 (Uno D8-D11). These are **outputs** – the MCU actively drives them, one at a time.
2. **Columns.** Wire keypad C1-C4 to PD2-PD5 (Uno D2-D5). These are **inputs** with the AVR's internal pull-up enabled – no external resistor, exactly like Lab 4-1's buttons.
3. **Display.** Wire segments a-f through 330 Ω resistors to PC0-PC5 (A0-A5), segment g through a 330 Ω resistor to PD6 (D6), and the common cathode straight to **GND**. One digit, always lit when driven – no multiplexed refresh loop this time.

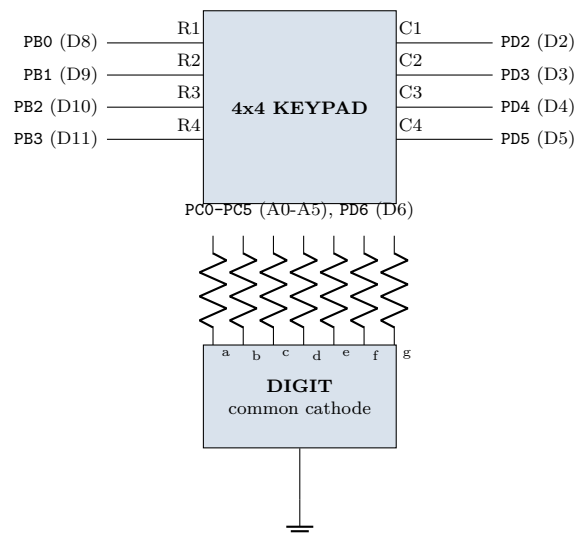


Figure 1: Keypad and single-digit display wiring. Each key merely bridges its row line to its column line when pressed – there is no component inside the keypad beyond that mechanical bridge. Segment resistors shown individually since, unlike Lab 4-1, there is only one digit to drive (no shared bus, no multiplexing).

Quick Experiments (Try It)

Try It 1: Rows and columns, by hand

Before writing any scanning code, drive PB0 (Row 1) **LOW** and PB1–PB3 **HIGH**, with columns configured as instructed above. Touch a jumper wire between the R1 and C1 pins on the breadboard (simulating pressing the “1” key) and confirm IN R17, PIND shows bit 2 (PD2, column 1) go **LOW** only while the wire bridges those two specific pins. This confirms your wiring before any scanning loop exists.

Try It 2: One row, no full scan

Write a short loop that drives *only* R1 low (R2–R4 stay high) and polls C1–C4. Confirm that keys 1, 2, 3, A register while keys on other rows (e.g. 5 or 9) do *not* – even though you have not yet written code to scan those other rows. This is your checkpoint that a single row’s scan works in isolation, the same incremental approach Lab 4-1 used before combining two digits.

In-Lab Exercises (target: 3 hours)

In-Lab Exercise 1: Wire and verify the display alone (30 min)

Wire the display per the instructions above and re-run Lab 4-1’s static-digit test (cycle 0-9, holding each briefly) *completely unchanged* except for segment g’s pin (PD2 → PD6). Confirm every digit 0-9 still renders correctly before wiring the keypad – you are only relocating one pin, not redesigning the display logic.

In-Lab Exercise 2: Single-row scan (35 min)

Repeat Try It 2 as a graded checkpoint: scan R1 only, and light the Uno’s on-board LED (PB5) – or show it on the display as a placeholder – whenever any of C1–C4 reads low. Confirm the LED responds *only* to keys 1, 2, 3, A and stays off for every other key on the pad.

In-Lab Exercise 3: Full 4x4 scan and decode (90 min)

Extend Exercise 2 into a loop that cycles all four rows. On detecting a pressed key, decode its (row, column) pair into a key value using the Quick Reference map above, and display it: 0–9 through DIGIT_TO_SEG (reused unchanged from Lab 4-1), anything else as a dash (segment g only).

- Add a software debounce using your t_{debounce} from the “Your Individual Target” box: after first detecting a key, wait t_{debounce} and confirm the same key still reads pressed before accepting it.
- **Trace/predict first, then verify:** before running it, predict what your program displays if two keys in *different* rows and *different* columns are held down at the same time (e.g. 2 and 8). Then try it and compare – does your prediction match, and can you explain any mismatch from how the scan visits rows in order?

In-Lab Exercise 4: Hold last key until the next press (25 min)

Confirm your debounce value matches your calculated t_{debounce} exactly, then extend Exercise 3 so the display holds the *last* key pressed persistently – not just for an instant – until a new key is pressed and debounced. Describe in your notebook what the display does differently

with versus without the debounce check from Exercise 3 (e.g. does the digit ever flicker or jump to a wrong value on a real keypress?).

AI Collaboration

Try these prompts with an AI tool:

- “How do I scan a 4x4 matrix keypad in AVR assembly using internal pull-up resistors on the columns?”
- “Why does a cheap matrix keypad without diodes suffer from ‘ghost’ keypresses when multiple keys are held down at once?”
- “What’s a clean way in assembly to convert a (row, column) pair into a single key value, given the keys aren’t in a simple row-times-four-plus- column order on every keypad?”

Critical check – verify, don’t just accept: Before trusting any decoded output, verify your *physical* keypad’s actual row/column/key layout against the Quick Reference table above – don’t assume an AI tool’s row/column map matches your specific keypad, since wiring order varies between manufacturers even for keypads that look identical.

Know the limit: AI tools will often hand you scanning code that quietly assumes a diode-protected keypad capable of full N-key rollover. Most classroom kits are diode-less, so holding multiple keys at once can produce a “phantom” key that was never pressed – a real hardware limitation the AI’s example code won’t warn you about unless you ask specifically.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Decode a scan cycle by hand

A scan cycle drives R1, R2, R3, R4 low in turn. While R3 is low, IN R17,PIND reads 0b11110111 (bits 2-5 are columns C1-C4 in that order, bit 2 = C1). Every other row’s read shows all four column bits high. Using the Quick Reference key map, which key was pressed?

Find the Bug 1: Pull-ups left disabled

A student wired the columns to PD2-PD5 exactly as instructed, correctly set DDRD bits 2-5 to 0 (input), but forgot to set the corresponding PORTD bits to 1 – so the internal pull-ups never turn on. No key is being pressed. Predict what IN R17,PIND reads on these four bits, and explain why an unconnected (floating) digital input does not reliably read as either a clean 0 or a clean 1.

Write Code on Paper 1: Hand-write the decode chain

On paper, without a computer, simulator, or AI tool, write the CPI/BRNE comparison chain (or equivalent) that decodes a scan result for *row R2 only* into its four key values (4, 5, 6, B) – i.e. given that R2 was the row just driven low, and one column bit is known to read low, write the branches that turn “column 1 is low” into displaying 4, “column 2 is low” into 5, and so on.

Draw on Paper 1: Draw the internal matrix

Draw the keypad's internal 4x4 grid from memory: 4 horizontal row lines, 4 vertical column lines, and a switch symbol at each of the 16 crossings, labeled with its key. Then, for the key 6, trace and highlight the exact current path from PB1 (R2, driven low) through the pressed switch to PD4 (C3, read low) – and explain, from your drawing, why no *other* column reads low while only the 6 key is held.

Reflection

In one short paragraph, compare this lab's keypad scanning to Lab 4's display multiplexing. Both drive one line at a time in a repeating cycle – but one is scanning *output* (lighting a digit) and the other is scanning *input* (reading a key). What stayed structurally the same between them (the one-at-a-time timing, the loop-over-four-lines shape), and what's fundamentally different (which direction data flows, and why this lab's scan rate has no flicker-fusion deadline the way Lab 4's display refresh did)?